

Code Jam 2009 - Round 1C

Analysis: All Your Base

```
IN A.D. 2101
WAR WAS BEGINNING
...
CATS: HOW ARE YOU GENTLEMEN !!
CATS: ALL YOUR BASE ARE BELONG TO US
```

This problem, of course, takes place in the halcyon days of A.D. 2100, before all our base became belong to Cats. It also firmly demonstrates that its author still lives in [A.D. 2001](#).

In this problem our job is to read a series of symbols and interpret them as digits of a number in some base. In order to *minimize* the number, we'll want to use the smallest base possible. We can check how many different characters show up in the number; if that number of characters is k , then we can work in base $\max(k, 2)$ (base 1 is not allowed). From there, it's simply a matter of assigning values from 0 to k to all of the characters. We can't start with a 0, so we should make the first digit a 1; and after that, simply assign the lowest number available to characters from left to right. In that way, "ab2ac999" becomes "10213444" in base 5, or 85499 seconds.

The following is a brief solution in Python:

```
import sys

N = int(sys.stdin.readline().strip())
for qw in range(1, N+1):
    print 'Case #{}:'.format(qw),

    num = sys.stdin.readline().strip()
    values = {num[0]: 1}
    for c in num:
        if c not in values:
            sz = len(values)
            if sz == 1:
                values[c] = 0
            else:
                values[c] = sz
    result = 0
    base = max(len(values), 2)
    for c in num:
        result *= base
        result += values[c]
    print result
```

One final note: on our official [Google Group](#), user Damien pointed out the following:

"The first example case in All Your Base implies the alien language uses a left-right notation. If that assumption is wrong and they actually used right-left you'd be 54 seconds late for the war: 11001001 binary = 201 decimal, but the reverse 10010011 is 147 decimal. Dangerous assumption to make don't you think? :-)"

I hate to say it, but he's right; even though 2219 people solved this problem, we may still be taken by surprise!

Analysis: Bribe the Prisoners

This problem is solved by dynamic programming. For each pair of cells $a \leq b$, we want to compute $dp[a][b]$, the best answer if we only have prisoners in cells from a to b , inclusive. Once we decide the location of c , the first prisoner between a and b to be released, we face the smaller sub-problems $dp[a][c-1]$ and $dp[c+1][b]$. The final answer we want is $dp[1][P]$.

It is clear we only need to solve those $dp[a][b]$'s where both a and b are either 1, P , or adjacent to a prisoner to be released. Thus the number of sub-problems we need to solve is just $O(Q^2)$.

Here is the annotated judge's solution.

```
int p[200]; // prisoners to be released.
map<pair<int, int>, int> dp;

// Finds the minimum amount of gold needed,
// if we only consider the cells from a to b, inclusive.
int Solve(int a, int b) {
    // First, look up the cache to see if the
    // result is computed before.
    pair<int, int> pr(a, b);
    if(mp.find(pr) != mp.end()) return mp[pr];

    // Start the computation.
    int r = 0;
    for(int i=0; i<Q; i++) {
        if(p[i] >= a && p[i] <= b) {
            int tmp = (b-a) + Solve(a, p[i]-1) + Solve(p[i]+1, b);
            if (!r || tmp<r) r=tmp;
        }
    }
    mp[pr]=r;
    return r;
}
```

Analysis: Center of Mass

Let P_i and V_i be the initial position and the velocity of the i -th firefly; its position at time t is given by $P_i + tV_i$. Therefore, the center of mass at time t is

$$M(t) = \sum (P_i + t V_i) / n = \sum P_i / n + t \sum V_i / n = P_{\text{ave}} + t V_{\text{ave}},$$

where P_{ave} ($= M(0)$) and V_{ave} are the average initial position and average speed. Thus we show that the center of mass is also moving on a straight ray (a half line) with constant speed. (One special case is that the average speed could be 0, when the center of mass does not move at all.)

So, the problem is actually finding the closest distance from a point (the origin) to a ray, an elementary geometry problem. This is an easy exercise with many possible short solutions. One can use basic manipulations or calculus to derive the exact formula for the answers. One may also notice that the distance function $d(t)$ is convex in t , thus use trinary search to find the best t . As always, we encourage the readers to download correct solutions from the scoreboard.

Note

We did the calculations using vectors. It can also be carried out with 3-dimensional coordinates.

Reference

The exact formula of the [distance from a point to a line in 3d space](#).